

Termination Checking

Ivan Wei

CSE590

General recursion is not compatible with dependent type theory

- Recall: no general recursion with dependent types

General recursion is not compatible with dependent type theory

- Recall: no general recursion with dependent types
- Problem 1: Infinitely recursive terms have any type; all propositions have a proof

General recursion is not compatible with dependent type theory

- Recall: no general recursion with dependent types
- Problem 1: Infinitely recursive terms have any type; all propositions have a proof
- Problem 2: $e_1 \equiv e_2$ needs to be decidable for identity types

Dependent type theory uses recursors

- Solution: Introduce limited recursion in the form of recursors

Example

$\Gamma \vdash \text{natind} :$	$(P : \mathbb{N} \rightarrow \text{Type})$	Proposition
	$\rightarrow P(Z)$	Base Case
	$\rightarrow ((x : \mathbb{N}) \rightarrow P(x) \rightarrow P(S(x)))$	IH
	$\rightarrow (n : \mathbb{N}) \rightarrow P(n)$	Result

$$\text{natind } P \ z \ s \ Z \equiv z$$

$$\text{natind } P \ z \ s \ S(n) \equiv s \ n \ (\text{natind } P \ z \ s \ n)$$

Central Questions

- How do we express more kinds of terminating recursive algorithms?
- How do we express recursive algorithms more easily?

Generalized Recursor (Nordström, 1988)

Nordström introduces a recursor on accessible values.

Definition

Let $\prec$ be a binary relation on a set A . $a \in A$ is said to be **accessible** provided that all descending chains $a \succ a_0 \succ \dots$ terminate. The set of all accessible elements of A is denoted $\text{Acc}(A, \prec)$

Definition

A is said to be **well-founded** provided that all its elements are accessible

Remark

If b is accessible and $a \prec b$, then a is also accessible.

Remark

Types can be viewed as sets that terms belong to.

Definition

Formalizing the remark on the previous page:

$$\Gamma \vdash \text{acc_helper} : (b : \text{Acc}(A, \prec)) \rightarrow (a : A) \rightarrow (a \prec b) \rightarrow \text{Acc}(A, \prec)$$

Definition

Nordström's general recursor is given by (slightly paraphrasing)

$$\begin{aligned} \Gamma \vdash \text{rec} : & (P : A \rightarrow \text{Type}) \\ & \rightarrow ((r : \text{Acc}(A, \prec)) \rightarrow ((q : A) \rightarrow (q \prec r) \rightarrow P(q)) \rightarrow P(r)) \\ & \rightarrow (p : \text{Acc}(A, \prec)) \\ & \rightarrow P(p) \end{aligned}$$

$$\text{rec } P \ e \ p \equiv e \ p \ (\text{fun } q \rightarrow \text{fun } pf \rightarrow \text{rec } P \ e \ (\text{acc_helper } p \ pf \ q))$$

- Evaluation: Nordström shows that `natind`, `listind` are special cases of his recursor
- Limitations: `p` and `e` are unwieldy to provide for initial call

Inductive Definition Syntax (Paulin-Mohring, 1993)

Instead of a single recursor form, allow for defining custom inductive definitions.

Definition

Inductive definitions have the syntax $\text{Ind}(X : A)\{C_1 \mid \cdots \mid C_n\}$, where A is an “arity”, and C_i ’s are constructors. They have type A .

Example

$$\text{vec} \equiv \text{Ind}(X : \mathbb{N} \rightarrow \text{Set})\{(X \ 0) \mid (n : \mathbb{N}) \rightarrow A \rightarrow (X \ n) \rightarrow (X \ (S \ n))\}$$

has type $\mathbb{N} \rightarrow \text{Set}$

Definition

Eliminators have the syntax $\text{Elim}(c, Q)\{f_1 | \dots | f_n\}$ where c is constructed through inductive definition, Q is some proposition on output, and f_i 's correspond to the constructors.

Example

If c is type $(\text{listn } n)$, Q is type $\mathbb{N} \rightarrow A$, x is type $(Q\ 0)$, and f is type $(n : \mathbb{N}) \rightarrow A \rightarrow (Q\ n) \rightarrow (Q\ (S\ n))$, then: $\text{Elim}(c, Q)\{x | f\}$ is type $(Q\ n)$

- Evaluation: Paulin-Mohring shows that this calculus with inductive definitions is strongly normalizing
- Limitations: Unwieldy to understand and use, and some expression issues

Structural Recursion (Abel and Altenkirch, 2000)

- Problem: Recursors are unwieldy, and programmers are used to regular recursion
- Observation: Most recursive algorithms are comprised of a case analysis followed by recursive call on case result

Example

```
fun x ->  
match x with  
| half Z = Z  
| half (S Z) = Z  
| half (S (S n)) = S (half n)
```

Definition

Structural ordering is broadly given as follows: for a value v and constructor C , $v < C(\dots, v, \dots)$

Example

```
fun x ->  
  match x with  
  | half Z = Z  
  | half (S Z) = Z  
  | half (S (S n)) = S (half n)
```

In this algorithm, $n < x$.

Remark

Suppose all types are well-founded under structural ordering. Suppose a function f only ever calls itself on structurally smaller arguments. Consider the sequence of calls with arguments v_i

$$f\ v_0, f\ v_1, f\ v_2 \dots$$

where

$$v_0 > v_1 > v_2 > \dots$$

Since no infinite descending chains of elements exist in well-founded sets, this sequence of calls must terminate.

- Evaluation: The authors provide a calculus and prove all values are accessible in the calculus.
- Limitations: Some terminating recursive functions are not accepted because the ordering judgements are only defined on elimination forms

Handling Mutual Recursion

Definition

In a **call matrix** (a_{ij}), the entry a_{ij} represents the relation $<$, $\leq$, or $?$ between callee argument i and caller argument j .

Definition

Addition on symbols is defined as combining information about a relation in parallel, while multiplication on symbols is defined as combining information about a relation serially.

$+$	$<$	$\leq$	$?$
$<$	$<$	$<$	$<$
$\leq$	$<$	$\leq$	$\leq$
$?$	$<$	$\leq$	$?$

$\cdot$	$<$	$\leq$	$?$
$<$	$<$	$<$	$?$
$\leq$	$<$	$\leq$	$?$
$?$	$?$	$?$	$?$

Matrix multiplication of call matrices then allows you to see argument relations across function composition!

Definition

A **call graph** is a directed graph where vertices represent functions and edges represent calls, and are labeled with a corresponding edge matrix.

To handle mutual recursion, saturate the call graph by performing all possible compositions until no new edges can be made. Then look at self calls.

Accessibility Proposition in Structural Recursion (Bove and Capretta, 2005)

- Problem: Some general recursive algorithms are not structurally recursive.
- Solution: Force these algorithms to be structurally recursive on termination proofs

Example

Consider the quicksort function:

```
quicksort: list nat -> list nat
quicksort nil = nil
quicksort cons(hd, tl) =
  let less = quicksort (filter_less hd tl) in
  let geq  = quicksort (filter_geq  hd tl) in
  append less (cons hd geq)
```


Example

We know: quicksort terminates on list xs when empty or when its recursive calls terminate. Can express this as a proposition:

$$qsAcc : list \mathbb{N} \rightarrow Set$$

Which has two constructors

$$\begin{aligned} nilAcc &: qsAcc \ nil \\ consAcc &: (hd : \mathbb{N}) \\ &\rightarrow (tl : list \ \mathbb{N}) \\ &\rightarrow qsAcc \ (filter_less \ \dots) \\ &\rightarrow qsAcc \ (filter_geq \ \dots) \\ &\rightarrow qsAcc \ (hd :: tl) \end{aligned}$$

Example

Expand quicksort definition to take in a proof of `qsAcc`

```
quicksort: (xs : list nat) -> qsAcc xs -> list nat
quicksort nil nilAcc = nil
quicksort cons(hd, tl) consAcc(hd, tl, pf1, pf2) =
  let less = quicksort (filter_less hd tl) pf1 in
  let geq  = quicksort (filter_geq  hd tl) pf2 in
  append less (cons hd geq)
```

Separately provide all calls to quicksort with the proof of `qsAcc` for all `xs`.

- Evaluation: Defines the set of recursive functions that can be expressed in dependent type theory this way, and a translation algorithm.
- Limitations: Still limited in kinds of recursive functions that can be expressed

Sized Types (Barthe, et. al, 2004)

- Problem: Structural recursion is limited in the kinds of functions it can express and termination checkers often suffer from implementation issues
- Solution: Index types by an additional size annotation τ^l , and adjust inference rules to ensure termination. s represents a size, and $\hat{s}$ represents the successor of that size.
- The authors define calculus $\hat{\lambda}$

Definition

$$\frac{}{\Gamma \vdash c : \text{Inst}_c^s \vec{\tau} \rightarrow d^{\hat{s}} \vec{\tau}} \text{ Constructor}$$
$$\frac{\Gamma \vdash e' : d^{\hat{s}} \vec{\tau} \quad \Gamma \vdash e_i : \text{Inst}_{c_i}^s \vec{\tau} \rightarrow \theta (1 \leq i \leq n)}{\Gamma \vdash \text{case } e' \text{ of } \{c_1 \Rightarrow e_1 \mid \cdots \mid c_n \Rightarrow e_n\} : \theta} \text{ Case}$$

Definition

$$\frac{\Gamma, f : d^{\iota} \vec{\tau} \rightarrow \theta \vdash e : d^{\hat{\iota}} \vec{\tau} \rightarrow [\hat{\iota}/\iota] \theta}{\Gamma \vdash (\text{letrec } f = e) : d^s \vec{\tau} \rightarrow [s/i] \theta} \text{ Rec}$$

$$\frac{\Gamma \vdash e : \tau^{\iota}}{\Gamma \vdash e : \tau^{\hat{\iota}}} \text{ Subsumption}$$

- Evaluation: The authors prove strong normalization in this calculus and show it is strictly more expressive than structural recursion.
- Limitations: $\hat{\lambda}$ doesn't have any dependent types

Calculus of Algebraic Constructions with Sized Types (Blanqui, 2004)

- Problem: $\hat{\lambda}$ needs to be scaled up to more complex calculi and worked out with rewrites
- Solution: Calculus of Algebraic Constructions with Size Annotations (CACSA)
- Addresses rewrites, ensures rewriting terminates
- Makes the assumption that rewrites preserve type
- Types are annotated by size expressions from a well-ordered *size algebra*
- C^a is a subtype of C^b when $a \leq b$
- Evaluation: Proves strong normalization,
- Limitations: Mostly technical, no “conditional rewrites” and “existential size variables”.

Conclusion

- Recursors $\rightarrow$ structural recursion for easier usability
- Structural recursion $\rightarrow$ sized types for easier implementation and more flexible expression of algorithms
- Future work is mostly centered around more technical expansion of sized types approach, like high rank size polymorphism, applications to universe checking, shape irrelevance, size irrelevance (Abel, et. al 2017).

References I



Andreas Abel and Thorsten Altenkirch.

A predicative analysis of structural recursion.

Journal of Functional Programming, 12, 12 2000.



Andreas Abel, Andrea Vezzosi, and Theo Winterhalter.

Normalization by evaluation for sized dependent types.

Proc. ACM Program. Lang., 1(ICFP), August 2017.



G. Barthe, M. J. Frade, E. Giménez, L. Pinto, and T. Uustalu.

Type-based termination of recursive definitions.

Mathematical Structures in Computer Science, 14(1):97–141, 2004.

References II



Frédéric Blanqui.

A type-based termination criterion for dependently-typed higher-order rewrite systems.

In Vincent van Oostrom, editor, *Rewriting Techniques and Applications*, pages 24–39, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.



Ana Bove and Venanzio Capretta.

Modelling general recursion in type theory.

Mathematical Structures in Computer Science, 15(4):671–708, 2005.



Bengt Nordström.

Terminating general recursion.

BIT Numerical Mathematics, 28(3):605–619, Sep 1988.

References III



Christine Paulin-Mohring.

Inductive definitions in the system coq - rules and properties.

In *International Conference on Typed Lambda Calculus and Applications*, 1993.